

Table of contents

Recherche Tabou 1

 Algorithme 1

 Formules clés 1

 Caractéristiques 1

 Paramètres de guidage 1

Recuit Simulé 1

 Algorithme 1

 Standard 1

 Parallèle 1

 Formules 1

 Caractéristiques 1

ACO 1

 Algorithme 1

 Formules 2

 Caractéristiques 2

Particle Swarm Optimization (PSO) 2

Firefly Algorithm 2

Cuckoo Search 2

Algorithmes Évolutionnaires 2

 Algorithme 2

 Formules Mathématiques 2

 Sélection proportionnelle 2

 Théorème des schémas 2

 Sélection par rang linéaire 2

 Caractéristiques 2

 Espace de recherche 2

 Convergence 2

 Guided Parameters 2

Genetic Programming (GP) 2

 Algorithme 2

 Formules 2

 Caractéristiques 2

Performance 2

 Challenges 2

 Formules Clés 2

 Évaluation 2

 Informations Essentielles 3

Transitions de phase en optimisation 3

 Challenges 3

 Formules clés 3

 Performance 3

 Métriques 3

 Régimes 3

 Informations essentielles 3

Recherche Tabou

Algorithme

Étapes :

- 1. **Initialiser** une solution x_0 , liste taboue vide T , paramètres (M, k)
- 2. **Tant que** critère d'arrêt non atteint :
 - Générer voisinage $V(x_c)$
 - **Choisir** le meilleur $x' \in V(x_c)$ **non-tabou** (ou avec aspiration)
 - $x_c \leftarrow x'$
 - **Mettre à jour** T (ajouter mouvement inverse, FIFO ou temps k)
 - (Optionnel) Mettre à jour mémoire long terme
- 3. **Retourner** meilleure solution trouvée

Formules clés

Fitness QAP :

$$f = \sum_{i,j} f_{ij} \cdot d_{r_i r_j}$$

Évaluation voisin :

$$\Delta = f(x') - f(x)$$

Condition tabou (exemple) : mouvement (r, s) interdit si

$$T_{i_s r} > t \quad \text{et} \quad T_{i_r s} > t$$

où T est la matrice tabou, t l'itération actuelle.

Caractéristiques

Espace de recherche : Combinatoire (permutations, graphes, etc.)

Convergence : Conditionnelle (si espace fini, voisinage symétrique, accessibilité), mais temps exponentiel possible.

Mémoire : Court terme (liste taboue FIFO/tenure k) + long terme (fréquences des mouvements).

Aspiration : Outrepasser tabou si solution meilleure que toutes précédentes.

Paramètres de guidage

- Taille liste taboue (M) : contrôle mémoire court terme
- Temps de bannissement (k) : durée d'interdiction d'un mouvement
- Structure du voisinage : type et nombre de mouvements
- Critère d'aspiration : condition pour ignorer tabou

Recuit Simulé

Algorithme

Standard

- 1. Initialiser solution x , température T
- 2. Tant que critère non atteint :
 - Générer voisin x'
 - $\Delta E = E(x') - E(x)$
 - Si $\Delta E \leq 0$: $x \leftarrow x'$
 - Sinon : $x \leftarrow x'$ avec proba $e^{-\Delta E/T}$
 - Diminuer T selon planning

Parallèle

- 1. Lancer M replicas à températures fixes $T_1 < \dots < T_M$
- 2. Parallèlement :
 - Chaque replica évolue indépendamment
 - Échanges périodiques entre replicas adjacents selon $p = \min(1, e^{-\Delta_{ij}})$

Formules

$$p_{\text{accept}} = \min(1, e^{-\Delta E/T})$$

$$T_{k+1} = \alpha T_k \quad (0 < \alpha < 1)$$

$$\Delta_{ij} = (E_i - E_j)(1/T_j - 1/T_i)$$

Caractéristiques

Espace : Continu ou discret

Convergence : Probabiliste vers optimum global si $T(t) \propto C/\log(t)$

Paramètres :

- T_0 (température initiale)
- Planning refroidissement (α , itérations/palier)
- Critère d'arrêt
- Mouvements élémentaires

ACO

Algorithme

AS (Ant System) :

- 1. Initialiser les phéromones
- 2. Chaque fourmi construit une solution via choix probabilistes basés sur phéromones τ et heuristique η
- 3. Évaluer les solutions
- 4. Évaporer les phéromones : $\tau \leftarrow (1 - \rho)\tau$
- 5. Déposer des phéromones sur les bonnes solutions
- 6. Répéter 2-5

- ACS (Ant Colony System) différences :
- Règle de décision : avec proba q_0 , choix déterministe ($\max [\tau][\eta]^\beta$) ; sinon règle AS
 - Mise à jour locale immédiate de l'évaporation après chaque choix
 - Seule la meilleure fourmi globale dépose des phéromones

Formules

Règle de décision AS :

$$p_{ij} = \frac{[\tau_{ij}]^\alpha [\eta_{ij}]^\beta}{\sum [\tau_{il}]^\alpha [\eta_{il}]^\beta}$$

Mise à jour phéromones AS :

$$\tau_{ij} \leftarrow (1 - \rho)\tau_{ij} + \sum_k \Delta\tau_{ij}^k$$

avec $\Delta\tau_{ij}^k = Q/L_k$ si arête dans solution k

Caractéristiques

Espace recherche : Combinatoire, construction incrémentale

Convergence : Asymptotique, risque de stagnation, équilibre exploration/exploitation critique

Paramètres guidés :

- AS : α (phéromone), β (heuristique), ρ (évaporation), m (#fourmis), Q
- ACS : + q_0 (seuil déterministe), ϕ (évaporation locale)

Valeurs typiques TSP : $\alpha = 1, \beta = 5, \rho = 0.5, m = n, Q = 1$

Particle Swarm Optimization (PSO)

Algorithme

1. Initialiser essaim aléatoire
2. Pour chaque particule, mettre à jour vitesse et position
3. Mettre à jour meilleures positions personnelle et globale
4. Répéter

Formules

$$\mathbf{v}_i = \omega \mathbf{v}_i + c_1 r_1 (\mathbf{x}_i^{\text{best}} - \mathbf{x}_i) + c_2 r_2 (\mathbf{B} - \mathbf{x}_i)$$

$$\mathbf{x}_i = \mathbf{x}_i + \mathbf{v}_i$$

Caractéristiques

- Espace : Continu (extensions discrètes)
- Convergence : Rapide
- Guided parameters : ω, c_1, c_2

Firefly Algorithm

Algorithme

1. Initialiser population aléatoire
2. Pour chaque paire (i,j), si $I_i < I_j$, déplacer i vers j
3. Mettre à jour intensités
4. Répéter

Formules

$$\beta = \beta_0 \exp(-\gamma r_{ij}^2)$$

$$\mathbf{x}_i = \mathbf{x}_i + \beta (\mathbf{x}_j - \mathbf{x}_i) + \alpha \epsilon$$

Caractéristiques

- Espace : Continu (extensions discrètes)
- Convergence : Bonne, équilibre via γ
- Guided parameters : β_0, γ, α

Cuckoo Search

Algorithme

1. Initialiser n nids aléatoires
2. Générer nouvelle solution par Lévy flight
3. Remplacer si meilleure
4. Abandonner fraction p_a des pires nids
5. Répéter

Formules

$$\mathbf{x}_i^{\text{new}} = \mathbf{x}_i + \alpha \oplus \text{Lévy}(\lambda)$$

Caractéristiques

- Espace : Continu
- Convergence : Rapide, bonne exploration
- Guided parameters : p_a, α, λ

Algorithmes Évolutionnaires

Algorithme

1. Initialiser population aléatoire
2. Évaluer fitness de chaque individu
3. Sélectionner les parents (par roulette, rang ou tournoi)
4. Appliquer croisement avec probabilité p_c
5. Appliquer mutation avec probabilité p_m
6. Former nouvelle génération avec élitisme
7. Répéter 2-6 jusqu'à critère d'arrêt

Formules Mathématiques

Sélection proportionnelle

$$p_i = \frac{f_i}{\sum_{j=1}^n f_j}$$

Théorème des schémas

$$M(t+1, S) \geq \frac{M(t, S)f(S, t)}{F(t)} \left[1 - p_e \frac{\delta(S)}{m-1} - o(S)p_m \right]$$

Sélection par rang linéaire

$$p_i = \frac{1}{n} \left[\beta - 2(\beta - 1) \frac{i-1}{n-1} \right]$$

Caractéristiques

Espace de recherche

- Discret, continu, permutations
- Encodage : binaire, réel, permutation
- Fonction boîte noire (pas de gradient)

Convergence

- Convergence globale (évite minima locaux)
- Diminishing returns (plateau en fin)
- Probabiliste

Guided Parameters

- n : taille population (20-200)
- p_c : probabilité croisement (0.6-0.9)
- p_m : probabilité mutation (0.001-0.1)
- Élitisme : 1-10%
- Pression sélection (β ou k tournoi)

Genetic Programming (GP)

Algorithme

1. Initialiser population de programmes aléatoires via F et T
2. Évaluer chaque programme avec fonction fitness (ex: erreur absolue)
3. Sélectionner meilleurs individus (tournoi/roulette)
4. Appliquer opérateurs :
 - Crossover : échanger sous-arbres entre parents
 - Mutation : remplacer sous-arbre aléatoire
5. Remplacer ancienne génération
6. Répéter 2-5 jusqu'à critère d'arrêt

Formules

Fitness régression :

$$f(p) = \sum_{i=1}^k |y_i - p(x_i)|$$

Caractéristiques

Espace : Composition infinie de F et T (arbres variables)

Convergence : Vers optimum local, sensible au surapprentissage et bloat

Paramètres :

- F (fonctions), T (terminaux)
- Taille population (50-1000)
- Probabilités crossover (0.8-0.95) et mutation (0.05-0.2)
- Méthode sélection (tournoi)
- Limite profondeur (5-20)
- Critère arrêt (générations/stagnation)

Performance

Challenges

- Stochasticité $\rightarrow$ résultats variables
- Pas de métrique universelle
- Dépendant des paramètres
- Dépendant du problème
- "No Free Lunch" : aucun algorithme n'est meilleur sur tous les problèmes

Formules Clés

Effort de calcul :

$$T(N) \propto N^\alpha$$

Taux de succès :

$$P_c = \frac{\text{solutions dans } \epsilon}{\text{runs}}$$

Théorème NFL :

$$\sum_f P(d_m \mid f, m, A_1) = \sum_f P(d_m \mid f, m, A_2)$$

Évaluation

Métriques :

- Effort : temps ou évaluations
- Qualité : erreur relative ou P_e
- Évolutivité : croissance avec N

Protocole :

- Moyenne sur 100-1000 runs

- Tests statistiques pour comparaison
- Benchmarks réels et synthétiques

Informations Essentielles

- Analyse empirique et statistique
- Paramètres critiques à régler
- Aucune méthode universelle
- Adapter l'algorithme au problème

Transitions de phase en optimisation

Challenges

- Complexité passe brutalement de linéaire à exponentielle
- Solutions deviennent rares quand $\alpha = M/N$ augmente
- Performances dépendent fortement du régime (α)

Formules clés

- Paramètre : $\alpha = M/N$
- 2-XORSAT : $\alpha_c = 0.5$
- 3-XORSAT : $\alpha_c = 0.9179$
- RWSAT : transition à $\alpha_E = 0.33$

Performance

Métriques

- P_{SAT} : probabilité de satisfaisabilité
- $\langle T_{res} \rangle$: temps moyen de résolution
- $e = E/M$: énergie normalisée

Régimes

- **Linéaire** ($\alpha < \alpha_E$) : $T \sim N$
- **Exponentiel** ($\alpha_E \leq \alpha < \alpha_c$) : $T \sim \exp(N)$
- **Insoluble** ($\alpha > \alpha_c$) : $P_{SAT} \rightarrow 0$

Informations essentielles

1. **Seuils critiques** :
 - 2-XORSAT : 0.5
 - 3-XORSAT : 0.9179
 - RWSAT : 0.33
 - Backtracking : 0.667
2. **Structure solutions** :
 - $\alpha < 0.818$: solutions uniformes
 - $\alpha \in [0.818, 0.918]$: solutions en clusters
 - $\alpha > 0.918$: aucune solution
3. **Recommandations** :
 - Monitorer α
 - Changer d'algorithme selon le régime
 - Anticiper explosion exponentielle